

Problem1

May 3, 2026

0.1 2.1 Cart-pole swing-up with limited actuation

(a)

The convex approximation (LOCP) around the current iterate $(s^{(i)}, u^{(i)})$ is

$$\begin{aligned} \text{minimize}_{s, u} \quad & (s_N - s_{\text{goal}})^\top P (s_N - s_{\text{goal}}) \\ & + \sum_{k=0}^{N-1} [(s_k - s_{\text{goal}})^\top Q (s_k - s_{\text{goal}}) + u_k^\top R u_k] \\ \text{subject to} \quad & s_0 = \bar{s}_0 \\ & u_k \in \mathcal{U}, \quad \forall k \in \{0, \dots, N-1\} \\ & \|u_k - u_k^{(i)}\|_\infty \leq \rho, \quad \forall k \in \{0, \dots, N-1\} \\ & \|s_k - s_k^{(i)}\|_\infty \leq \rho, \quad \forall k \in \{0, \dots, N\} \\ & s_{k+1} = A_k s_k + B_k u_k + c_k, \quad \forall k \in \{0, \dots, N-1\} \end{aligned}$$

where the terminal constraint $s_N = s_{\text{goal}}$ is replaced by the terminal cost $(s_N - s_{\text{goal}})^\top P (s_N - s_{\text{goal}})$, and A_k, B_k, c_k come from the linearization as follow:

$$s_{k+1} = f_{\mathbf{d}}(s_k^{(i)}, u_k^{(i)}) + \frac{\partial f_{\mathbf{d}}}{\partial s}(s_k^{(i)}, u_k^{(i)})(s_k - s_k^{(i)}) + \frac{\partial f_{\mathbf{d}}}{\partial u}(s_k^{(i)}, u_k^{(i)})(u_k - u_k^{(i)}) = A_k s_k + B_k u_k + c_k$$

with

$$A_k = \frac{\partial f_{\mathbf{d}}}{\partial s}(s_k^{(i)}, u_k^{(i)}), \quad B_k = \frac{\partial f_{\mathbf{d}}}{\partial u}(s_k^{(i)}, u_k^{(i)}), \quad c_k = f_{\mathbf{d}}(s_k^{(i)}, u_k^{(i)}) - A_k s_k^{(i)} - B_k u_k^{(i)}$$

(b) Please see `cartpole_swingup_constrained.py`.

(c)

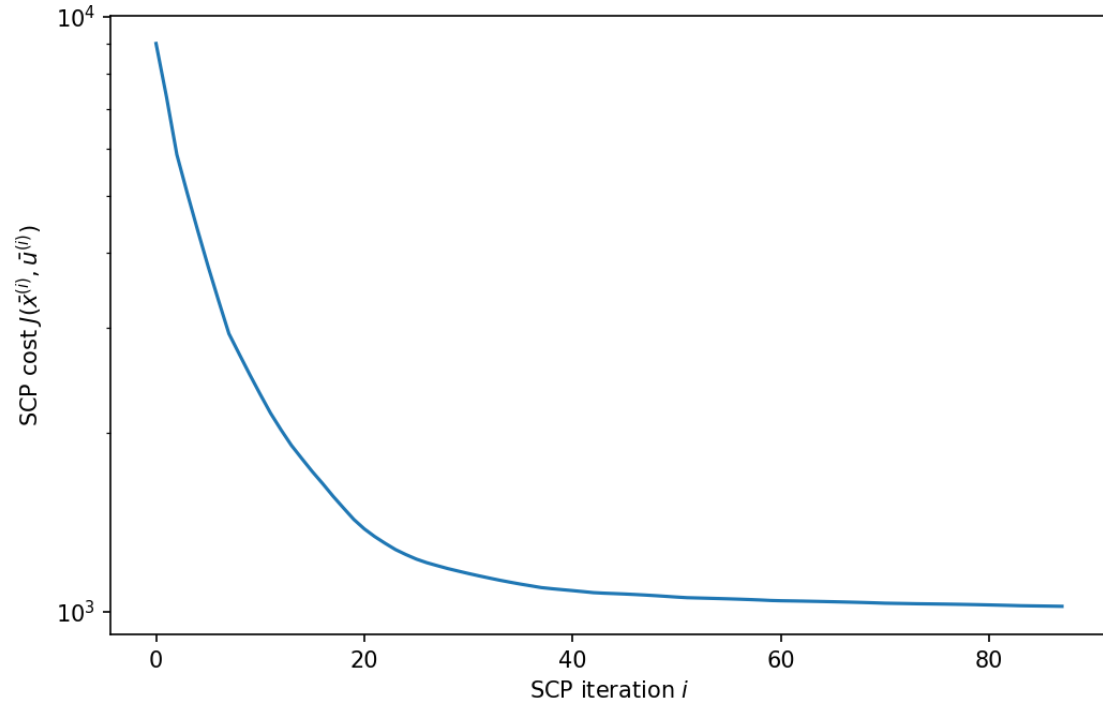
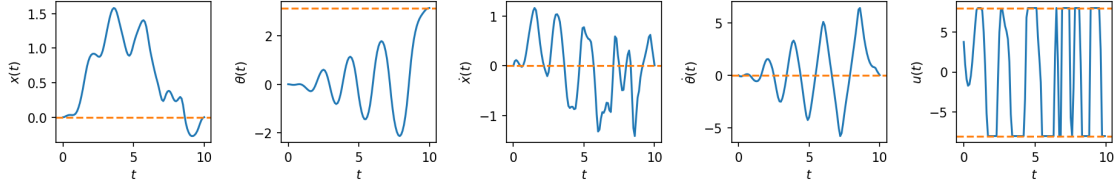
Please see `cartpole_swingup_constrained.py`.

(d)

```
[1]: %run cartpole_swingup_constrained.py
```

```
88%|          | 88/100 [01:21<00:11, 1.08it/s, objective change=0.32265]
```

SCP converged after 88 iterations.



The pendulum just reaches s_{goal} at the end of the horizon but does not stay there. A natural scheme is to use SCP to generate the swing-up trajectory, then switch to a closed-loop LQR controller linearized around the upright equilibrium to keep the pendulum there indefinitely.

Problem2

May 3, 2026

0.1 2.2 Shortest path through a grid

(a)

Doing DP backward (B to A) we compute at each node the optimal cost-to-go (J^* in the figure) and store the optimal edge to follow, represented as a blue arrow. Finally, the optimal path from A to B is computed by always choosing the optimal edge and is highlighted in red. It corresponds to: **up, up, down, up, down, down**.

```
[1]: from IPython.display import Image, display
display(Image(filename='/Users/erwinpoussi/Downloads/NBMetadataCache 2.png',
↪width=700))
```

on a grid. Consider the shortest path problem in a grid. A path is a sequence of nodes $x_0, x_1, \dots, x_n$ such that $x_{i+1} \leftarrow \text{right and top}$ and $x_{i-1} \leftarrow \text{left and bottom}$. The cost of a path is the sum of the edge weights. The decision to go to a node x depends on the cost of the path to x and the cost of the path from x to the goal node. Given a two-dimensional state space, we will use dynamic programming to find the shortest path from node A to node B .

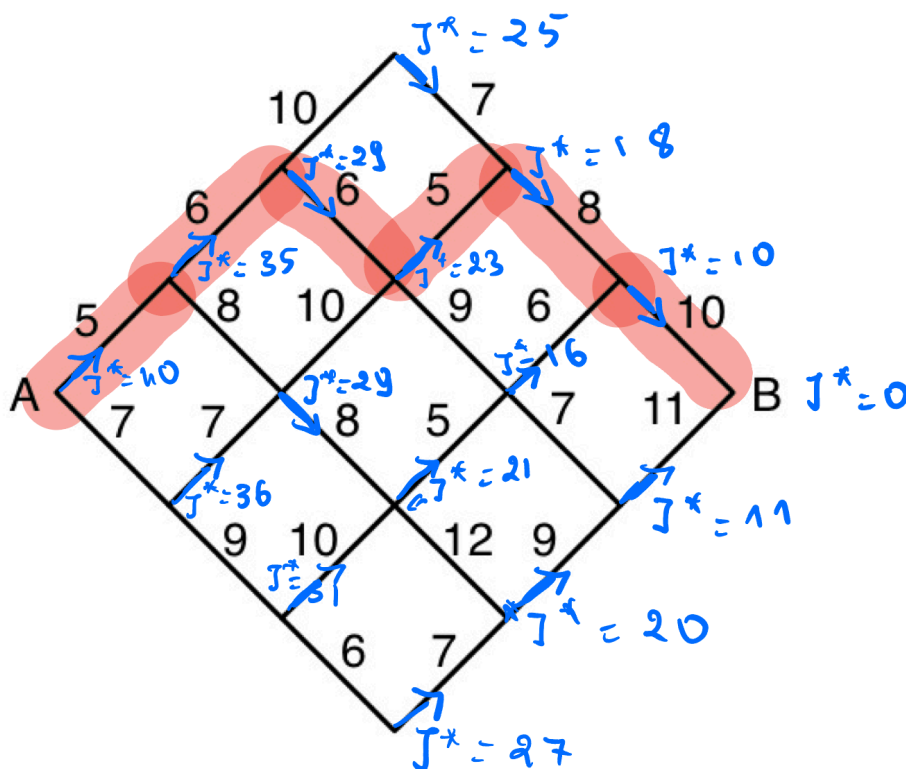


Figure 1: Shortest path problem on a grid.

Dynamic Programming (DP) to find the shortest path from A to B . The grid is a diamond shape with 5 columns of nodes. The nodes are labeled with their cost to reach from A. The shortest path is highlighted with blue arrows and red shading. The path starts at A (cost 5), goes to the top node (cost 10), then to the top-right node (cost 7), then to the bottom-right node (cost 11), then to the bottom node (cost 6), and finally to B (cost 0). The total cost of the shortest path is 27.

(b)

Number of nodes. For a grid with n segments per side, the diamond has columns $0, 1, \dots, 2n$,

with the number of nodes per column growing from 1 to $n + 1$ then shrinking back to 1:

$$n_{\text{nodes}} = \sum_{k=0}^n (k+1) + \sum_{k=1}^n k = \frac{(n+1)(n+2)}{2} + \frac{n(n+1)}{2} = (n+1)^2.$$

For $n = 3$: $n_{\text{nodes}} = 16$.

Exhaustive search. Every path from A to B consists of $2n$ steps, each either up or down, and must end at B (same row as A), which requires exactly n ups and n downs. The number of such paths is therefore

$$N_{\text{exhaustive}} = \binom{2n}{n} = \frac{(2n)!}{(n!)^2}.$$

For $n = 3$: $\binom{6}{3} = 20$ routes.

Dynamic programming. The DP backward sweep evaluates the cost-to-go J^* once at every node except the terminal B . At each node the computation is a single min over at most 2 successors, so the total number of DP evaluations equals the number of non-terminal nodes:

$$N_{\text{DP}} = (n+1)^2 - 1 = n^2 + 2n.$$

For $n = 3$: $N_{\text{DP}} = 9 + 6 = 15$ evaluations.

Comparison. $\binom{2n}{n}$ grows exponentially ($\sim 4^n / \sqrt{\pi n}$) while $n^2 + 2n$ grows only polynomially, illustrating the considerable computational advantage of DP over exhaustive search.

problem3

May 8, 2026

0.1 2.3 Cart-pole balance

(a)

Starting from the continuous-time dynamics $\dot{s} = f(s, u)$, we apply **Euler integration** to get the discrete-time approximation:

$$s_{k+1} \approx s_k + \Delta t f(s_k, u_k).$$

We then **linearize** f around the equilibrium $(\bar{s}, \bar{u})$ via a first-order Taylor expansion:

$$f(s_k, u_k) \approx f(\bar{s}, \bar{u}) + \left. \frac{\partial f}{\partial s} \right|_{(\bar{s}, \bar{u})} (s_k - \bar{s}) + \left. \frac{\partial f}{\partial u} \right|_{(\bar{s}, \bar{u})} (u_k - \bar{u}).$$

Since $(\bar{s}, \bar{u})$ is an equilibrium, $f(\bar{s}, \bar{u}) = 0$, and $\bar{u} = 0$. Defining $\tilde{s}_k = s_k - \bar{s}$ and subtracting $\bar{s}$ from both sides:

$$\tilde{s}_{k+1} \approx \tilde{s}_k + \Delta t \left(\left. \frac{\partial f}{\partial s} \right|_{(\bar{s}, \bar{u})} \tilde{s}_k + \left. \frac{\partial f}{\partial u} \right|_{(\bar{s}, \bar{u})} u_k \right) = \underbrace{\left(I + \Delta t \left. \frac{\partial f}{\partial s} \right|_{(\bar{s}, \bar{u})} \right)}_A \tilde{s}_k + \underbrace{\Delta t \left. \frac{\partial f}{\partial u} \right|_{(\bar{s}, \bar{u})}}_B u_k.$$

Substituting the given Jacobians evaluated at $\bar{s} = (0, \pi, 0, 0)$, $\bar{u} = 0$:

$$A = I_4 + \Delta t \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & \frac{m_p g}{m_c} & 0 & 0 \\ 0 & \frac{(m_c + m_p)g}{m_c \ell} & 0 & 0 \end{bmatrix}, \quad B = \Delta t \begin{bmatrix} 0 \\ 0 \\ \frac{1}{m_c} \\ \frac{1}{m_c \ell} \end{bmatrix}.$$

(b)

```
[1]: import numpy as np
from cartpole_balance import compute_lti_matrices, ricatti_recursion

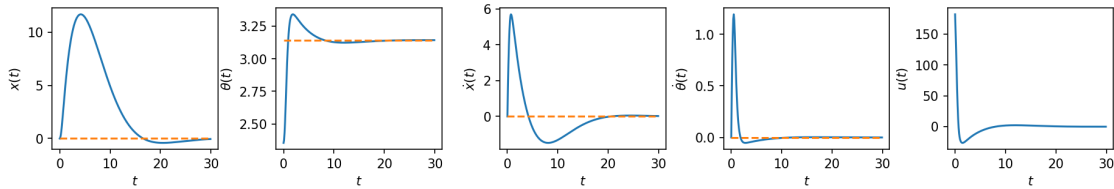
A, B = compute_lti_matrices()
Q, R = np.eye(4), np.eye(1)
K = ricatti_recursion(A, B, Q, R)
print("K_inf:\n", np.round(K, 2))
```

```
K: [[ 0.7291397 -231.85419273  4.21967187 -68.24742822]]
K_inf:
[[ 0.73 -231.85  4.22 -68.25]]
```

(c)

```
[2]: from cartpole_balance import simulate, plot_state_and_control_history

t = np.arange(0.0, 30.0, 0.1)
s_ref = np.array([0.0, np.pi, 0.0, 0.0]) * np.ones((t.size, 1))
u_ref = np.array([0.0])
s0 = np.array([0.0, 3 * np.pi / 4, 0.0, 0.0])
s, u = simulate(t, s_ref, u_ref, s0, K)
plot_state_and_control_history(s, u, t, s_ref, "cartpole_balance")
```



```
[3]: from animations import animate_cartpole
from IPython.display import HTML
import matplotlib.pyplot as plt

fig, ani = animate_cartpole(t, s[:, 0], s[:, 1])
plt.close(fig)
HTML(ani.to_jshtml())
```

[3]: <IPython.core.display.HTML object>

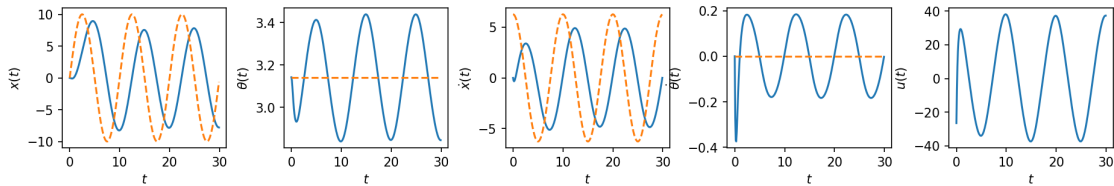
(d)

(i) The Jacobians $\frac{\partial f}{\partial s}$ and $\frac{\partial f}{\partial u}$ do not depend on the cart position x — only on θ , $\dot{\theta}$, and u . Since the reference trajectory always maintains $\bar{\theta}(t) = \pi$, $\dot{\bar{\theta}}(t) = 0$, and $\bar{u} = 0$, the linearization point is identical at every time step regardless of the time-varying $\bar{x}(t)$. Therefore A and B are constant and do not need to be recomputed.

(iii) The desired trajectory $\bar{x}(t) = a \sin(2\pi t/T)$ requires the cart to accelerate ($\ddot{x} \neq 0$), which physically means a nonzero reference force $\bar{u}(t) \neq 0$ is needed to drive it. The LQR was designed assuming $\bar{u} = 0$, so there is no feedforward term to generate this force. As a result the controller cannot simultaneously track $\bar{x}(t)$ and keep $\theta = \pi$: the cart acceleration perturbs the pendulum, causing θ and $\dot{\theta}$ to oscillate. Increasing Q tightens regulation around the reference but does not fix the missing feedforward, so θ and $\dot{\theta}$ keep oscillating.

```
[4]: from cartpole_balance import simulate, reference, plot_state_and_control_history

t = np.arange(0.0, 30.0, 0.1)
s_ref = np.array([reference(ti) for ti in t])
u_ref = np.array([0.0])
s0 = np.array([0.0, np.pi, 0.0, 0.0])
s, u = simulate(t, s_ref, u_ref, s0, K)
plot_state_and_control_history(s, u, t, s_ref, "cartpole_balance_tv")
```



```
[5]: from animations import animate_cartpole
from IPython.display import HTML
import matplotlib.pyplot as plt

fig, ani = animate_cartpole(t, s[:, 0], s[:, 1])
plt.close(fig)
HTML(ani.to_jshtml())
```

```
[5]: <IPython.core.display.HTML object>
```

```
[ ]:
```


problem4

May 3, 2026

0.1 2.4 Cart-pole swing-up

(a) Please see `cartpole_swingup.py`.

(b)

Using a second order Taylor expansion of the stage cost $c(\tilde{s}_k, \tilde{u}_k)$ around $(\bar{s}_k, \bar{u}_k)$:

$$c(\tilde{s}_k, \tilde{u}_k) = c(\bar{s}_k, \bar{u}_k) + \underbrace{\left(\frac{\partial c}{\partial s} \Big|_{(\bar{s}_k, \bar{u}_k)} \right)}_{=q_k}^\top \tilde{s}_k + \underbrace{\left(\frac{\partial c}{\partial u} \Big|_{(\bar{s}_k, \bar{u}_k)} \right)}_{=r_k}^\top \tilde{u}_k + \underbrace{\frac{1}{2} \tilde{s}_k^\top \frac{\partial^2 c}{\partial s^2} \Big|_{(\bar{s}_k, \bar{u}_k)}}_{=Q} \tilde{s}_k + \underbrace{\frac{1}{2} \tilde{u}_k^\top \frac{\partial^2 c}{\partial u^2} \Big|_{(\bar{s}_k, \bar{u}_k)}}_{=R} \tilde{u}_k$$

hence $\boxed{q_k = Q(\bar{s}_k - s_{\text{goal}})}$ and $\boxed{r_k = R\bar{u}_k}$.

Same for the terminal cost $c_N(\tilde{s}_N)$:

$$c_N(\tilde{s}_N) = c_N(\bar{s}_N) + \underbrace{\left(\frac{\partial c_N}{\partial s} \Big|_{\bar{s}_N} \right)}_{=q_N}^\top \tilde{s}_N + \underbrace{\frac{1}{2} \tilde{s}_N^\top \frac{\partial^2 c_N}{\partial s^2} \Big|_{\bar{s}_N}}_{=Q_N} \tilde{s}_N$$

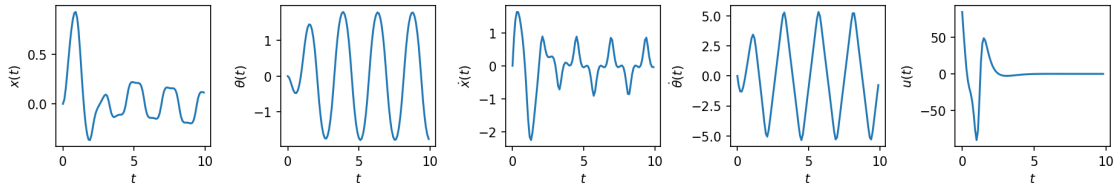
hence $\boxed{q_N = Q_N(\bar{s}_N - s_{\text{goal}})}$.

(c) Please see `cartpole_swingup.py`.

(d) Please see `cartpole_swingup.py`.

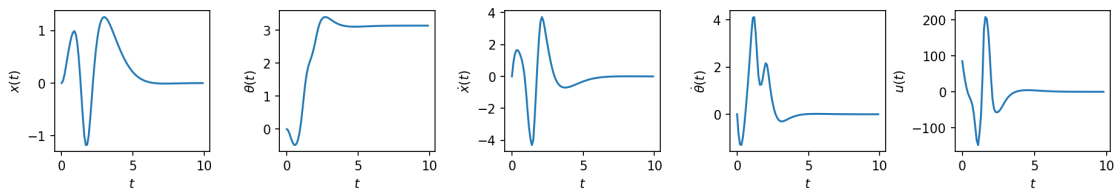
```
[1]: from IPython.display import Image, display
print("Open-loop:")
display(Image(filename='cartpole_swingup_ol.png'))
```

Open-loop:



```
[2]: print("Closed-loop:")
      display(Image(filename='cartpole_swingup_cl.png'))
```

Closed-loop:



While the open-loop is consistently oscillating, the closed-loop is progressively damped towards convergence as expected.

Problem5

May 3, 2026

0.1 2.5 Machine maintenance

States: R (running), B (broken) at the start of each week.

Terminal: $J_5(R) = J_5(B) = 0$

Immediate expected net profits and transitions:

- R + Maintain: $0.6(100 - 20) + 0.4(0 - 20) = 40$, next state: R w.p. 0.6, B w.p. 0.4
- R + No maintain: $0.3(100) + 0.7(0) = 30$, next state: R w.p. 0.3, B w.p. 0.7
- B + Repair: $0.6(100 - 40) + 0.4(0 - 40) = 20$, next state: R w.p. 0.6, B w.p. 0.4
- B + Replace: $1(100 - 150) = -50$, next state: R w.p. 1

Week 4:

$J_4(R)$: Maintain $\rightarrow 40$, No maintain $\rightarrow 30 \Rightarrow J_4(R) = \max(40, 30) = \mathbf{40}$ (Maintain)

$J_4(B)$: Repair $\rightarrow 20$, Replace $\rightarrow -50 \Rightarrow J_4(B) = \max(20, -50) = \mathbf{20}$ (Repair)

Week 3:

$J_3(R)$: Maintain $\rightarrow 40 + 0.6(40) + 0.4(20) = 72$, No maintain $\rightarrow 30 + 0.3(40) + 0.7(20) = 56$
 $\Rightarrow J_3(R) = \max(72, 56) = \mathbf{72}$ (Maintain)

$J_3(B)$: Repair $\rightarrow 20 + 0.6(40) + 0.4(20) = 52$, Replace $\rightarrow -50 + 40 = -10 \Rightarrow J_3(B) = \max(52, -10) = \mathbf{52}$ (Repair)

Week 2:

$J_2(R)$: Maintain $\rightarrow 40 + 0.6(72) + 0.4(52) = 104$, No maintain $\rightarrow 30 + 0.3(72) + 0.7(52) = 88$
 $\Rightarrow J_2(R) = \max(104, 88) = \mathbf{104}$ (Maintain)

$J_2(B)$: Repair $\rightarrow 20 + 0.6(72) + 0.4(52) = 84$, Replace $\rightarrow -50 + 72 = 22 \Rightarrow J_2(B) = \max(84, 22) = \mathbf{84}$ (Repair)

Week 1 (new machine, guaranteed to run $\Rightarrow$ earns \$100, ends in state R, no maintain needed):

$$J_1 = 100 + J_2(R) = 100 + 104 = \boxed{204}$$

cartpole_balance.py

cartpole_balance.py

```

1  """
2  Solution code for the problem "Cart-pole balance".
3
4  Autonomous Systems Lab (ASL), Stanford University
5  """
6
7  import numpy as np
8  from scipy.integrate import odeint
9  import matplotlib.pyplot as plt
10
11 from animations import animate_cartpole
12
13 # Constants
14 n = 4 # state dimension
15 m = 1 # control dimension
16 mp = 2.0 # pendulum mass
17 mc = 10.0 # cart mass
18 L = 1.0 # pendulum length
19 g = 9.81 # gravitational acceleration
20 dt = 0.1 # discretization time step
21 animate = False # whether or not to animate results
22
23
24 def cartpole(s: np.ndarray, u: np.ndarray) -> np.ndarray:
25     """Compute the cart-pole state derivative
26
27     Args:
28         s (np.ndarray): The cartpole state: [x, theta, x_dot, theta_dot], shape (n,)
29         u (np.ndarray): The cartpole control: [F_x], shape (m,)
30
31     Returns:
32         np.ndarray: The state derivative, shape (n,)
33     """
34     x, theta, dx, dtheta = s
35     sintheta, costheta = np.sin(theta), np.cos(theta)
36     h = mc + mp * (sintheta**2)
37     ds = np.array(
38         [
39             dx,
40             dtheta,
41             (mp * sintheta * (L * (dtheta**2) + g * costheta) + u[0]) / h,
42             -((mc + mp) * g * sintheta + mp * L * (dtheta**2) * sintheta * costheta + u[0] * costheta)
43             / (h * L),
44         ]
45     )
46     return ds
47
48
49 def reference(t: float) -> np.ndarray:
50     """Compute the reference state (s_bar) at time t
51
52     Args:
53         t (float): Evaluation time
54
55     Returns:
56         np.ndarray: Reference state, shape (n,)
57     """
58     a = 10.0 # Amplitude
59     T = 10.0 # Period
60
61     # PART (d) #####
62     return np.array([
63         a * np.sin(2 * np.pi * t / T),
64         np.pi,
65         a * (2 * np.pi / T) * np.cos(2 * np.pi * t / T),
66         0.0,
67     ])
68     # END PART (d) #####
69
70
71 def ricatti_recursion(

```

```

72 A: np.ndarray, B: np.ndarray, Q: np.ndarray, R: np.ndarray
73 ) -> np.ndarray:
74 """Compute the gain matrix K through Ricatti recursion
75
76 Args:
77     A (np.ndarray): Dynamics matrix, shape (n, n)
78     B (np.ndarray): Controls matrix, shape (n, m)
79     Q (np.ndarray): State cost matrix, shape (n, n)
80     R (np.ndarray): Control cost matrix, shape (m, m)
81
82 Returns:
83     np.ndarray: Gain matrix K, shape (m, n)
84 """
85 eps = 1e-4 # Riccati recursion convergence tolerance
86 max_iters = 1000 # Riccati recursion maximum number of iterations
87 P_prev = np.zeros((n, n)) # initialization
88 converged = False
89 for i in range(max_iters):
90     # PART (b) #####
91     K = -np.linalg.inv(R + B.T @ P_prev @ B) @ B.T @ P_prev @ A
92     P = Q + A.T @ P_prev @ (A + B @ K)
93     if np.max(np.abs(P - P_prev)) < eps:
94         converged = True
95         break
96     P_prev = P
97     # END PART (b) #####
98 if not converged:
99     raise RuntimeError("Ricatti recursion did not converge!")
100 print("K:", K)
101 return K
102
103
104 def simulate(
105     t: np.ndarray, s_ref: np.ndarray, u_ref: np.ndarray, s0: np.ndarray, K: np.ndarray
106 ) -> tuple[np.ndarray, np.ndarray]:
107     """Simulate the cartpole
108
109     Args:
110         t (np.ndarray): Evaluation times, shape (num_timesteps,)
111         s_ref (np.ndarray): Reference state s_bar, evaluated at each time t. Shape (num_timesteps, n)
112         u_ref (np.ndarray): Reference control u_bar, shape (m,)
113         s0 (np.ndarray): Initial state, shape (n,)
114         K (np.ndarray): Feedback gain matrix (Ricatti recursion result), shape (m, n)
115
116     Returns:
117         tuple[np.ndarray, np.ndarray]: Tuple of:
118             np.ndarray: The state history, shape (num_timesteps, n)
119             np.ndarray: The control history, shape (num_timesteps, m)
120     """
121
122     def cartpole_wrapper(s, t, u):
123         """Helper function to get cartpole() into a form preferred by odeint, which expects t as the second arg"""
124         return cartpole(s, u)
125
126     # PART (c) #####
127     N = t.size - 1
128     s = np.zeros((t.size, n))
129     u = np.zeros((t.size, m))
130     s[0] = s0
131     for k in range(N):
132         u[k] = K @ (s[k] - s_ref[k]) + u_ref
133         s[k + 1] = odeint(cartpole_wrapper, s[k], t[k : k + 2], (u[k],))[1]
134     u[N] = K @ (s[N] - s_ref[N]) + u_ref
135     # END PART (c) #####
136     return s, u
137
138
139 def compute_lti_matrices() -> tuple[np.ndarray, np.ndarray]:
140     """Compute the linearized dynamics matrices A and B of the LTI system
141
142     Returns:
143         tuple[np.ndarray, np.ndarray]: Tuple of:
144             np.ndarray: The A (dynamics) matrix, shape (n, n)
145             np.ndarray: The B (controls) matrix, shape (n, m)
146     """
147     # PART (a) #####

```

```

148     A = np.eye(n) + dt * np.array([
149         [0, 0, 1, 0],
150         [0, 0, 0, 1],
151         [0, mp * g / mc, 0, 0],
152         [0, (mc + mp) * g / (mc * L), 0, 0],
153     ])
154     B = dt * np.array([[0], [0], [1 / mc], [1 / (mc * L)]])
155     # END PART (a) #####
156     return A, B
157
158
159 def plot_state_and_control_history(
160     s: np.ndarray, u: np.ndarray, t: np.ndarray, s_ref: np.ndarray, name: str
161 ) -> None:
162     """Helper function for cartpole visualization
163
164     Args:
165         s (np.ndarray): State history, shape (num_timesteps, n)
166         u (np.ndarray): Control history, shape (num_timesteps, m)
167         t (np.ndarray): Times, shape (num_timesteps,)
168         s_ref (np.ndarray): Reference state s_bar, evaluated at each time t. Shape (num_timesteps, n)
169         name (str): Filename prefix for saving figures
170     """
171     fig, axes = plt.subplots(1, n + m, dpi=150, figsize=(15, 2))
172     plt.subplots_adjust(wspace=0.35)
173     labels_s = (r"$x(t)$", r"$\theta(t)$", r"$\dot{x}(t)$", r"$\dot{\theta}(t)$")
174     labels_u = (r"$u(t)$",)
175     for i in range(n):
176         axes[i].plot(t, s[:, i])
177         axes[i].plot(t, s_ref[:, i], "--")
178         axes[i].set_xlabel(r"$t$")
179         axes[i].set_ylabel(labels_s[i])
180     for i in range(m):
181         axes[n + i].plot(t, u[:, i])
182         axes[n + i].set_xlabel(r"$t$")
183         axes[n + i].set_ylabel(labels_u[i])
184     plt.savefig(f"{name}.png", bbox_inches="tight")
185     plt.show()
186
187     if animate:
188         fig, ani = animate_cartpole(t, s[:, 0], s[:, 1])
189         ani.save(f"{name}.mp4", writer="ffmpeg")
190         plt.show()
191
192
193 def main():
194     # Part A
195     A, B = compute_lti_matrices()
196
197     # Part B
198     Q = np.eye(n) # state cost matrix
199     R = np.eye(m) # control cost matrix
200     K = ricatti_recursion(A, B, Q, R)
201
202     # Part C
203     t = np.arange(0.0, 30.0, 1 / 10)
204     s_ref = np.array([0.0, np.pi, 0.0, 0.0]) * np.ones((t.size, 1))
205     u_ref = np.array([0.0])
206     s0 = np.array([0.0, 3 * np.pi / 4, 0.0, 0.0])
207     s, u = simulate(t, s_ref, u_ref, s0, K)
208     plot_state_and_control_history(s, u, t, s_ref, "cartpole_balance")
209
210     # Part D
211     # Note: t, u_ref unchanged from part c
212     s_ref = np.array([reference(ti) for ti in t])
213     s0 = np.array([0.0, np.pi, 0.0, 0.0])
214     s, u = simulate(t, s_ref, u_ref, s0, K)
215     plot_state_and_control_history(s, u, t, s_ref, "cartpole_balance_tv")
216
217
218 if __name__ == "__main__":
219     main()

```

cartpole_swingup.py


```

1  """
2  Starter code for the problem "Cart-pole swing-up".
3
4  Autonomous Systems Lab (ASL), Stanford University
5  """
6
7
8  import time
9
10
11 from animations import animate_cartpole
12
13 import jax
14 import jax.numpy as jnp
15
16 import matplotlib.pyplot as plt
17
18 import numpy as np
19
20
21 from scipy.integrate import odeint
22
23
24
25 def linearize(f, s, u):
26     """Linearize the function `f(s, u)` around `(s, u)`.
27
28     Arguments
29     -----
30     f : callable
31         A nonlinear function with call signature `f(s, u)`.
32     s : numpy.ndarray
33         The state (1-D).
34     u : numpy.ndarray
35         The control input (1-D).
36
37     Returns
38     -----
39     A : numpy.ndarray
40         The Jacobian of `f` at `(s, u)`, with respect to `s`.
41     B : numpy.ndarray
42         The Jacobian of `f` at `(s, u)`, with respect to `u`.
43     """
44     # WRITE YOUR CODE BELOW #####
45     # INSTRUCTIONS: Use JAX to compute `A` and `B` in one line.
46     A, B = jax.jacobian(f, argnums=(0, 1))(s, u)
47     #####
48     return A, B
49
50
51 def ilqr(f, s0, s_goal, N, Q, R, QN, eps=1e-3, max_iters=1000):
52     """Compute the iLQR set-point tracking solution.
53
54     Arguments
55     -----
56     f : callable

```

```

64     A function describing the discrete-time dynamics, such that
65     `s[k+1] = f(s[k], u[k])`.
66     s0 : numpy.ndarray
67         The initial state (1-D).
68     s_goal : numpy.ndarray
69         The goal state (1-D).
70     N : int
71         The time horizon of the LQR cost function.
72     Q : numpy.ndarray
73         The state cost matrix (2-D).
74     R : numpy.ndarray
75         The control cost matrix (2-D).
76     QN : numpy.ndarray
77         The terminal state cost matrix (2-D).
78     eps : float, optional
79         Termination threshold for iLQR.
80     max_iters : int, optional
81         Maximum number of iLQR iterations.
82
83     Returns
84     -----
85     s_bar : numpy.ndarray
86         A 2-D array where `s_bar[k]` is the nominal state at time step `k`,
87         for `k = 0, 1, ..., N-1`
88     u_bar : numpy.ndarray
89         A 2-D array where `u_bar[k]` is the nominal control at time step `k`,
90         for `k = 0, 1, ..., N-1`
91     Y : numpy.ndarray
92         A 3-D array where `Y[k]` is the matrix gain term of the iLQR control
93         law at time step `k`, for `k = 0, 1, ..., N-1`
94     y : numpy.ndarray
95         A 2-D array where `y[k]` is the offset term of the iLQR control law
96         at time step `k`, for `k = 0, 1, ..., N-1`
97
98     """
99     if max_iters <= 1:
100         raise ValueError("Argument `max_iters` must be at least 1.")
101     n = Q.shape[0] # state dimension
102     m = R.shape[0] # control dimension
103
104     # Initialize gains `Y` and offsets `y` for the policy
105     Y = np.zeros((N, m, n))
106     y = np.zeros((N, m))
107
108     # Initialize the nominal trajectory `(s_bar, u_bar)`, and the
109     # deviations `(ds, du)`
110     u_bar = np.zeros((N, m))
111     s_bar = np.zeros((N + 1, n))
112     s_bar[0] = s0
113     for k in range(N):
114         s_bar[k + 1] = f(s_bar[k], u_bar[k])
115     ds = np.zeros((N + 1, n))
116     du = np.zeros((N, m))
117
118     # iLQR loop

```

```

129     converged = False
130     for _ in range(max_iters):
131         # Linearize the dynamics at each step `k` of `(s_bar, u_bar)`
132         A, B = jax.vmap(linearize, in_axes=(None, 0, 0))(f, s_bar[:-1], u_bar)
133         A, B = np.array(A), np.array(B)
134
135         # PART (c) #####
136         # INSTRUCTIONS: Update `Y`, `y`, `ds`, `du`, `s_bar`, and `u_bar`.
137
138         # Backward pass: Riccati recursion for P (value Hessian), p (value gradient),
139         # gains Y[k] and offsets y[k]
140         P = QN
141         p = QN @ (s_bar[-1] - s_goal)
142         for k in range(N - 1, -1, -1):
143             inv = np.linalg.inv(R + B[k].T @ P @ B[k])
144             Y[k] = -inv @ B[k].T @ P @ A[k]
145             y[k] = -inv @ (R @ u_bar[k] + B[k].T @ p)
146             p = Q @ (s_bar[k] - s_goal) + A[k].T @ (p + P @ B[k] @ y[k])
147             P = Q + A[k].T @ P @ (A[k] + B[k] @ Y[k])
148
149         # Forward pass: apply policy on real dynamics
150         s_new = np.zeros_like(s_bar)
151         u_new = np.zeros_like(u_bar)
152         s_new[0] = s0
153         for k in range(N):
154             du[k] = Y[k] @ (s_new[k] - s_bar[k]) + y[k]
155             u_new[k] = u_bar[k] + du[k]
156             s_new[k + 1] = f(s_new[k], u_new[k])
157         s_bar = s_new
158         u_bar = u_new
159
160         #####
161
162         if np.max(np.abs(du)) < eps:
163             converged = True
164             break
165     if not converged:
166         raise RuntimeError("iLQR did not converge!")
167     return s_bar, u_bar, Y, y
168
169 def cartpole(s, u):
170     """Compute the cart-pole state derivative."""
171     mp = 2.0 # pendulum mass
172     mc = 10.0 # cart mass
173     L = 1.0 # pendulum length
174     g = 9.81 # gravitational acceleration
175
176     x, theta, dx, dtheta = s
177     sintheta, costheta = jnp.sin(theta), jnp.cos(theta)
178     h = mc + mp * (sintheta**2)
179     ds = jnp.array(
180         [
181             dx,
182

```

```

194         dθ,
195         (mp * sinθ * (L * (dθ**2) + g * cosθ) + u[0]) / h,
196         -((mc + mp) * g * sinθ + mp * L * (dθ**2) * sinθ * cosθ + u[0] * cosθ)
197         / (h * L),
198     ]
199 )
200 )
201 return ds
202
203
204
205 # Define constants
206 n = 4 # state dimension
207 m = 1 # control dimension
208 Q = np.diag(np.array([10.0, 10.0, 2.0, 2.0])) # state cost matrix
209 R = 1e-2 * np.eye(m) # control cost matrix
210 QN = 1e2 * np.eye(n) # terminal state cost matrix
211 s0 = np.array([0.0, 0.0, 0.0, 0.0]) # initial state
212 s_goal = np.array([0.0, np.pi, 0.0, 0.0]) # goal state
213 T = 10.0 # simulation time
214 dt = 0.1 # sampling time
215 animate = False # flag for animation
216 closed_loop = True # flag for closed-loop control
217
218 # Initialize continuous-time and discretized dynamics
219 f = jax.jit(cartpole)
220 fd = jax.jit(lambda s, u, dt=dt: s + dt * f(s, u))
221
222 # Compute the iLQR solution with the discretized dynamics
223 print("Computing iLQR solution ... ", end="", flush=True)
224 start = time.time()
225 t = np.arange(0.0, T, dt)
226 N = t.size - 1
227 s_bar, u_bar, Y, y = ilqr(fd, s0, s_goal, N, Q, R, QN)
228 print("done! ({:.2f} s)".format(time.time() - start), flush=True)
229
230 # Simulate on the true continuous-time system
231 print("Simulating ... ", end="", flush=True)
232 start = time.time()
233 s = np.zeros((N + 1, n))
234 u = np.zeros((N, m))
235 s[0] = s0
236 for k in range(N):
237     # PART (d) #####
238     # INSTRUCTIONS: Compute either the closed-loop or open-loop value of
239     # `u[k]`, depending on the Boolean flag `closed_loop`.
240     if closed_loop:
241         u[k] = u_bar[k] + Y[k] @ (s[k] - s_bar[k]) + y[k]
242     else: # do open-loop control
243         u[k] = u_bar[k]
244     #####
245     s[k + 1] = odeint(lambda s, t: f(s, u[k]), s[k], t[k : k + 2])[1]
246 print("done! ({:.2f} s)".format(time.time() - start), flush=True)
247
248 # Plot
249 fig, axes = plt.subplots(1, n + m, dpi=150, figsize=(15, 2))

```

```

plt.subplots_adjust(wspace=0.45)
labels_s = (r"$x(t)$", r"$\theta(t)$", r"$\dot{x}(t)$", r"$\dot{\theta}(t)$")
labels_u = (r"$u(t)$",)
for i in range(n):
    axes[i].plot(t, s[:, i])
    axes[i].set_xlabel(r"$t$")
    axes[i].set_ylabel(labels_s[i])
for i in range(m):
    axes[n + i].plot(t[:-1], u[:, i])
    axes[n + i].set_xlabel(r"$t$")
    axes[n + i].set_ylabel(labels_u[i])
if closed_loop:
    plt.savefig("cartpole_swingup_cl.png", bbox_inches="tight")
else:
    plt.savefig("cartpole_swingup_ol.png", bbox_inches="tight")
plt.show()

if animate:
    fig, ani = animate_cartpole(t, s[:, 0], s[:, 1])
    ani.save("cartpole_swingup.mp4", writer="ffmpeg")
    plt.show()

```

cartpole_swingup_constrained.py

```

1 """
2 Starter code for the problem "Cart-pole swing-up with limited actuation".
3
4
5 Autonomous Systems Lab (ASL), Stanford University
6 """
7
8 from functools import partial
9
10 from animations import animate_cartpole
11
12
13 import cvxpy as cvx
14
15 import jax
16 import jax.numpy as jnp
17
18
19 import matplotlib.pyplot as plt
20
21 import numpy as np
22
23
24 from tqdm import tqdm
25
26
27 @partial(jax.jit, static_argnums=(0,))
28 @partial(jax.vmap, in_axes=(None, 0, 0))
29 def affinize(f, s, u):
30     """Affinize the function `f(s, u)` around `(s, u)`.
31
32     Arguments
33     -----
34     f : callable
35         A nonlinear function with call signature `f(s, u)`.
36     s : numpy.ndarray
37         The state (1-D).
38     u : numpy.ndarray
39         The control input (1-D).
40
41     Returns
42     -----
43     A : jax.numpy.ndarray
44         The Jacobian of `f` at `(s, u)`, with respect to `s`.
45     B : jax.numpy.ndarray
46         The Jacobian of `f` at `(s, u)`, with respect to `u`.
47     c : jax.numpy.ndarray
48         The offset term in the first-order Taylor expansion of `f` at `(s, u)`
49         that sums all vector terms strictly dependent on the nominal point
50         `(s, u)` alone.
51     """
52     # PART (b) #####
53     # INSTRUCTIONS: Use JAX to affinize `f` around `(s, u)` in two lines.
54     A, B = jax.jacobian(f, argnums=(0, 1))(s, u)
55     c = f(s, u) - A@s - B@u
56     # END PART (b) #####
57     return A, B, c
58
59
60 def solve_swingup_scp(f, s0, s_goal, N, P, Q, R, u_max, p, eps, max_iters):
61     """Solve the cart-pole swing-up problem via SCP.
62
63     Arguments
64     -----
65     f : callable
66         A function describing the discrete-time dynamics, such that
67         `s[k+1] = f(s[k], u[k])`.
68     s0 : numpy.ndarray
69         The initial state (1-D).
70     s_goal : numpy.ndarray
71         The goal state (1-D).

```

```

81 N : int
82     The time horizon of the LQR cost function.
83 P : numpy.ndarray
84     The terminal state cost matrix (2-D).
85 Q : numpy.ndarray
86     The state stage cost matrix (2-D).
87 R : numpy.ndarray
88     The control stage cost matrix (2-D).
89 u_max : float
90     The bound defining the control set [-u_max, u_max].
91 p : float
92     Trust region radius.
93 eps : float
94     Termination threshold for SCP.
95 max_iters : int
96     Maximum number of SCP iterations.
97
98 Returns
99 -----
100 s : numpy.ndarray
101     A 2-D array where s[k] is the open-loop state at time step k,
102     for k = 0, 1, ..., N-1
103 u : numpy.ndarray
104     A 2-D array where u[k] is the open-loop state at time step k,
105     for k = 0, 1, ..., N-1
106 J : numpy.ndarray
107     A 1-D array where J[i] is the SCP sub-problem cost after the i-th
108     iteration, for i = 0, 1, ..., (iteration when convergence occurred)
109 """
110 n = Q.shape[0] # state dimension
111 m = R.shape[0] # control dimension
112
113 # Initialize dynamically feasible nominal trajectories
114 u = np.zeros((N, m))
115 s = np.zeros((N + 1, n))
116 s[0] = s0
117 for k in range(N):
118     s[k + 1] = fd(s[k], u[k])
119
120 # Do SCP until convergence or maximum number of iterations is reached
121 converged = False
122 J = np.zeros(max_iters + 1)
123 J[0] = np.inf
124 for i in (prog_bar := tqdm(range(max_iters))):
125     s, u, J[i + 1] = scp_iteration(f, s0, s_goal, s, u, N, P, Q, R, u_max, p)
126     dJ = np.abs(J[i + 1] - J[i])
127     prog_bar.set_postfix({"objective change": "{:.5f}".format(dJ)})
128     if dJ < eps:
129         converged = True
130         print("SCP converged after {} iterations.".format(i))
131         break
132 if not converged:
133     raise RuntimeError("SCP did not converge!")
134 J = J[1 : i + 1]
135 return s, u, J
136
137 def scp_iteration(f, s0, s_goal, s_prev, u_prev, N, P, Q, R, u_max, p):
138     """Solve a single SCP sub-problem for the cart-pole swing-up problem.
139
140     Arguments
141     -----
142     f : callable
143         A function describing the discrete-time dynamics, such that
144         s[k+1] = f(s[k], u[k]).
145     s0 : numpy.ndarray
146         The initial state (1-D).
147     s_goal : numpy.ndarray
148         The goal state (1-D).

```



```

163     s_prev : numpy.ndarray
164         The state trajectory around which the problem is convexified (2-D).
165     u_prev : numpy.ndarray
166         The control trajectory around which the problem is convexified (2-D).
167     N : int
168         The time horizon of the LQR cost function.
169     P : numpy.ndarray
170         The terminal state cost matrix (2-D).
171     Q : numpy.ndarray
172         The state stage cost matrix (2-D).
173     R : numpy.ndarray
174         The control stage cost matrix (2-D).
175     u_max : float
176         The bound defining the control set `[-u_max, u_max]`.
177     p : float
178         Trust region radius.
179
180     Returns
181     -----
182     s : numpy.ndarray
183         A 2-D array where `s[k]` is the open-loop state at time step `k`,
184         for `k = 0, 1, ..., N-1`
185     u : numpy.ndarray
186         A 2-D array where `u[k]` is the open-loop state at time step `k`,
187         for `k = 0, 1, ..., N-1`
188     J : float
189         The SCP sub-problem cost.
190
191     """
192     A, B, c = affinize(f, s_prev[:-1], u_prev)
193     A, B, c = np.array(A), np.array(B), np.array(c)
194     n = Q.shape[0]
195     m = R.shape[0]
196     s_cvx = cvx.Variable((N + 1, n))
197     u_cvx = cvx.Variable((N, m))
198
199     # PART (c) #####
200     # INSTRUCTIONS: Construct the convex SCP sub-problem.
201     objective = 0.0
202     constraints = []
203     constraints += [s_cvx[0] == s0]
204     constraints += [cvx.norm(s_cvx[k] - s_prev[k], 'inf') <= p for k in range(N+1)]
205     constraints += [cvx.norm(u_cvx[k] - u_prev[k], 'inf') <= p for k in range(N)]
206     constraints += [cvx.norm(u_cvx[k], 'inf') <= u_max for k in range(N)]
207     constraints += [s_cvx[k + 1] == A[k] @ s_cvx[k] + B[k] @ u_cvx[k] + c[k] for k in range(N)]
208     objective += cvx.quad_form(s_cvx[N] - s_goal, P)
209     objective += cvx.sum([cvx.quad_form(s_cvx[k] - s_goal, Q) + cvx.quad_form(u_cvx[k], R) for k in range(N)])
210     # END PART (c) #####
211
212     prob = cvx.Problem(cvx.Minimize(objective), constraints)
213     prob.solve()
214     if prob.status != "optimal":
215         raise RuntimeError("SCP solve failed. Problem status: " + prob.status)
216     s = s_cvx.value
217     u = u_cvx.value
218     J = prob.objective.value
219     return s, u, J
220
221 def cartpole(s, u):
222     """Compute the cart-pole state derivative."""
223     mp = 1.0 # pendulum mass
224     mc = 4.0 # cart mass
225     L = 1.0 # pendulum length
226     g = 9.81 # gravitational acceleration
227
228     x, theta, dx, dtheta = s
229     sintheta, costheta = jnp.sin(theta), jnp.cos(theta)
230     h = mc + mp * (sintheta**2)
231     ds = jnp.array(

```

```

245     [
246         dx,
247         dθ,
248         (mp * sinθ * (L * (dθ**2) + g * cosθ) + u[0]) / h,
249         -((mc + mp) * g * sinθ + mp * L * (dθ**2) * sinθ * cosθ + u[0] * cosθ)
250         / (h * L),
251     ]
252 ]
253 )
254
255 return ds
256
257
258 def discretize(f, dt):
259     """Discretize continuous-time dynamics `f` via Runge-Kutta integration."""
260
261     def integrator(s, u, dt=dt):
262         k1 = dt * f(s, u)
263         k2 = dt * f(s + k1 / 2, u)
264         k3 = dt * f(s + k2 / 2, u)
265         k4 = dt * f(s + k3, u)
266         return s + (k1 + 2 * k2 + 2 * k3 + k4) / 6
267
268     return integrator
269
270
271 # Define constants
272 n = 4 # state dimension
273 m = 1 # control dimension
274 s_goal = np.array([0, np.pi, 0, 0]) # desired upright pendulum state
275 s0 = np.array([0, 0, 0, 0]) # initial downright pendulum state
276 dt = 0.1 # discrete time resolution
277 T = 10.0 # total simulation time
278 P = 1e3 * np.eye(n) # terminal state cost matrix
279 Q = np.diag([1e-2, 1.0, 1e-3, 1e-3]) # state cost matrix
280 R = 1e-3 * np.eye(m) # control cost matrix
281 p = 1.0 # trust region parameter
282 u_max = 8.0 # control effort bound
283 eps = 5e-1 # convergence tolerance
284 max_iters = 100 # maximum number of SCP iterations
285 animate = False # flag for animation
286
287 # Initialize the discrete-time dynamics
288 fd = jax.jit(discretize(cartpole, dt))
289
290 # Solve the swing-up problem with SCP
291 t = np.arange(0.0, T + dt, dt)
292 N = t.size - 1
293 s, u, J = solve_swingup_scp(fd, s0, s_goal, N, P, Q, R, u_max, p, eps, max_iters)
294
295 # Simulate open-loop control
296 for k in range(N):
297     s[k + 1] = fd(s[k], u[k])
298
299 # Plot state and control trajectories
300 fig, ax = plt.subplots(1, n + m, dpi=150, figsize=(15, 2))
301 plt.subplots_adjust(wspace=0.45)
302 labels_s = (r"$x(t)$", r"$\theta(t)$", r"$\dot{x}(t)$", r"$\dot{\theta}(t)$")
303 labels_u = (r"$u(t)$",)
304 for i in range(n):
305     ax[i].plot(t, s[:, i])
306     ax[i].axhline(s_goal[i], linestyle="--", color="tab:orange")
307     ax[i].set_xlabel(r"$t$")
308     ax[i].set_ylabel(labels_s[i])
309 for i in range(m):
310     ax[n + i].plot(t[:-1], u[:, i])
311     ax[n + i].axhline(u_max, linestyle="--", color="tab:orange")
312     ax[n + i].axhline(-u_max, linestyle="--", color="tab:orange")
313     ax[n + i].set_xlabel(r"$t$")
314     ax[n + i].set_ylabel(labels_u[i])
315 plt.savefig("cartpole_swingup_constrained.png", bbox_inches="tight")

```

```
# Plot cost history over SCP iterations
fig, ax = plt.subplots(1, 1, dpi=150, figsize=(8, 5))
ax.semilogy(J)
ax.set_xlabel(r"SCP iteration $i$")
ax.set_ylabel(r"SCP cost $J(\bar{x}^{\{i\}}, \bar{u}^{\{i\}})$")
plt.savefig("cartpole_swingup_constrained_cost.png", bbox_inches="tight")
plt.show()

# Animate the solution
if animate:
    fig, ani = animate_cartpole(t, s[:, 0], s[:, 1])
    ani.save("cartpole_swingup_constrained.mp4", writer="ffmpeg")
    plt.show()
```